Relatório Técnico: Heurística Construtiva e Meta-heurística GRASP para o Caminho Hamiltoniano Mínimo

Tailan de Souza Oliveira Rubens Rabelo Soares
Curso de Sistemas de Informação – Universidade Federal do Ceará (Campus Quixadá)
Disciplina: Pesquisa Operacional – Prof. Dr. Paulo Henrique Macêdo

Outubro de 2026

1. Metodologia e Funcionamento dos Algoritmos

Este trabalho resolve o Problema do Caminho Hamiltoniano Mínimo (PCHM) em grafos direcionados com origem fixa no vértice $s = 1$ e destino fixo no vértice $t = n$. Os testes foram realizados em Julia 1.12.6 em um computador Linux x86_64 (Intel Core i5-12450HX com 12 threads e 15,34 GB de RAM DDR5), usando semente fixa (`seed = 42`) para garantir que os resultados possam ser reproduzidos.

Na Atividade 1, foi implementada a heurística construtiva inicial baseada na regra do vizinho mais próximo. O algoritmo começa no vértice 1 e, a cada passo, escolhe o vértice intermediário ainda não visitado que possui a viagem mais barata a partir do ponto atual. Quando todos os nós intermediários já foram visitados, o algoritmo liga o último vértice diretamente ao destino final n. Por olhar apenas para o próximo passo imediato, essa abordagem é muito rápida, mas costuma deixar conexões caras para o final do caminho.

Na Atividade 2, foi implementada a meta-heurística GRASP com busca local Or-Opt de 1 nó. Antes de fixar os parâmetros finais, realizamos uma bateria rigorosa de calibração conjunta em grade (*grid search*), variando sistematicamente o fator de aleatoriedade $\alpha \in \{0,00;\ 0,05;\ 0,15;\ 0,30;\ 0,50;\ 1,00\}$ e o número de iterações em cinco níveis (100, 250, 500, 750 e 1000) em todas as 13 instâncias da TSPLIB. Esse experimento evidenciou que a configuração $\alpha = 0,30$ com 1000 iterações alcança o melhor desempenho global de qualidade (menor erro médio em relação ao ótimo exato de Programação Linear Inteira, com $\text{Gap}_{\text{PLI}} = 3,51\%$), sendo adotada como padrão para a resolução do trabalho. Em cada iteração do GRASP, constrói-se um caminho partindo de $s = 1$ e sorteando o próximo vértice dentro da Lista Restrita de Candidatos (LRC), delimitada pelo limiar $c_{\min} + \alpha(c_{\max} - c_{\min})$. Para garantir a conectividade acíclica durante a construção, testamos e comparamos duas abordagens: uma utilizando a estrutura de conjuntos disjuntos (*Union-Find* com compressão de caminhos e união por *rank*) e outra baseada em marcação booleana direta de visitados. Por fim, a busca local por realocação remove sucessivamente cada vértice intermediário e testa sua reinserção nas demais posições entre 2 e $n-1$, aplicando imediatamente qualquer melhoria encontrada (*first improvement*) até a convergência para um ótimo local.

2. Resultados Computacionais e Comparação com o Modelo PLI

A Tabela 1 consolida os dados exportados nos arquivos CSV de resultados e compara os três métodos avaliados nas 13 instâncias da TSPLIB. Para facilitar a leitura da tabela e do arquivo CSV gerado pelo código (`resultado_otimizacao.csv`), cada coluna possui o seguinte significado direto: **Instância** é o nome do arquivo de teste da TSPLIB; n (ou `N`) é a quantidade total de vértices do grafo; $Z_{\textbf{Inicial}}$ e $T_{\textbf{Inicial}}$ são o custo total da rota e o tempo gasto (em segundos) pela Heurística Construtiva Inicial (Atividade 1); $Z_{\textbf{GRASP}}$ é o menor custo de rota encontrado pela meta-heurística GRASP após 1000 iterações (Atividade 2); $T_{\textbf{GRASP}}^{\textbf{Com UF}}$ e $T_{\textbf{GRASP}}^{\textbf{Sem UF}}$ representam o tempo de execução (em segundos) do GRASP usando e sem usar a estrutura *Union-Find*; **Melhoria (%)** indica a redução percentual de custo obtida pelo GRASP sobre a solução inicial, calculada por $\frac{Z_{\text{Inicial}} - Z_{\text{GRASP}}}{Z_{\text{Inicial}}} \times 100\%$; $Z_{\textbf{PLI}}$ **(TP1)** e $T_{\textbf{PLI}}$ são o custo ótimo (ou melhor solução) e o tempo em segundos obtidos pelo modelo exato de Programação Linear Inteira no Gurobi (limitado a 180s no Trabalho Prático 01); e $\textbf{Gap}_{\textbf{PLI}}$ **(%)** mede a distância percentual do resultado do GRASP em relação ao modelo exato, dada por $\frac{Z_{\text{GRASP}} - Z_{\text{PLI}}}{Z_{\text{PLI}}} \times 100\%$. No arquivo CSV, também são salvas as colunas `Caminho_Valido` (que confirma com `true` que a rota passou no validador sem subciclos) e `Rota_Percorrida` (listada na Seção 4).

Tabela 1: Comparação entre Heurística Inicial, Meta-heurística GRASP (1000 iterações, $\alpha = 0,30$) e Modelo Exato PLI (TP 01).

Instância	n	Z_{Inicial}	T_{Inicial} (s)	Z_{GRASP}	$T_{\text{GRASP}}^{\text{Com UF}}$ (s)	$T_{\text{GRASP}}^{\text{Sem UF}}$ (s)	Melhoria (%)	Z_{PLI} (TP1)	T_{PLI} (s)	Gap_{PLI} (%)
`br17.atsp`	17	56,00	$< 0,0001$	34,00	0,2786	0,2859	39,29%	34,00	0,32	0,00%
`ft53.atsp`	53	9583,00	$< 0,0001$	6472,00	11,4582	14,2820	32,46%	6196,00	12,37	4,45%
`ft70.atsp`	70	42057,00	$< 0,0001$	39733,00	36,9449	56,1217	5,53%	37655,00	2,06	5,52%
`ftv33.atsp`	34	1547,00	$< 0,0001$	1319,00	1,5176	2,0465	14,74%	1275,00	0,47	3,45%
`ftv35.atsp`	36	1750,00	$< 0,0001$	1454,00	1,9351	2,8102	16,91%	1443,00	0,35	0,76%
`ftv38.atsp`	39	1828,00	$< 0,0001$	1537,00	2,7867	4,1536	15,92%	1511,00	0,39	1,72%
`ftv44.atsp`	45	1933,00	$< 0,0001$	1706,00	4,7041	6,7497	11,74%	1614,00	0,95	5,70%
`ftv47.atsp`	48	2298,00	$< 0,0001$	1756,00	5,9205	8,4351	23,59%	1754,00	1,43	0,11%
`ftv55.atsp`	56	2052,00	$< 0,0001$	1698,00	11,1244	15,6459	17,25%	1613,00	2,06	5,27%
`ftv64.atsp`	65	2586,00	$< 0,0001$	1966,00	28,8957	43,1280	23,98%	1840,00	6,11	6,85%
`ftv70.atsp`	71	2637,00	$< 0,0001$	2134,00	44,7971	61,8545	19,07%	1958,00	6,89	8,99%
`p43.atsp`	43	642,00	$< 0,0001$	599,00	5,3440	6,6011	6,70%	589,00*	180,04	1,70%
`ry48p.atsp`	48	17971,00	$< 0,0001$	14225,00	8,9207	11,0807	20,84%	14070,00	1,70	1,10%

*Em `p43.atsp`, o modelo PLI atingiu o tempo limite de 180s sem provar otimalidade global, retornando a melhor solução encontrada (589, 00).

3. Análise dos Resultados e Principais Diferenças

Na comparação entre a Heurística Inicial e o GRASP calibrado ($\alpha = 0,30$, 1000 iterações), a principal diferença aparece na qualidade das rotas geradas. Embora a heurística inicial termine quase instantaneamente (menos de $0,0001$ s em todos os testes), ela toma decisões míopes ao escolher sempre a aresta mais barata do momento sem planejar a chegada ao destino n. O GRASP superou expressivamente o resultado inicial em todas as 13 instâncias, com uma redução média de custo de $19,08\%$. As maiores economias percentuais ocorreram em `br17.atsp` (queda de $56,00$ para $34,00$, economia de $39,29\%$), em `ft53.atsp` (queda de $9583,00$ para $6472,00$, ganho de $32,46\%$, superior aos $6950,00$ obtidos anteriormente com $\alpha = 0,15$) e em `ftv64.atsp` (redução de $23,98\%$). Mesmo em instâncias com menor margem de melhora, como `ft70.atsp` e `p43.atsp`, o GRASP reduziu os custos em $5,53\%$ e $6,70\%$.

Quando comparamos o GRASP com o modelo exato de Programação Linear Inteira (PLI) do Trabalho Prático 01, o GRASP alcançou o valor ótimo global ($\text{Gap}_{\text{PLI}} = 0,00\%$) em `br17.atsp` ($34,00$) e ficou praticamente empatado com o ótimo em diversas instâncias: `ftv47.atsp` com apenas $0,11\%$ de diferença ($1756,00$ contra $1754,00$), `ftv35.atsp` ($0,76\%$), `ry48p.atsp` ($1,10\%$), `p43.atsp` ($1,70\%$), `ftv38.atsp` ($1,72\%$) e `ftv33.atsp` ($3,45\%$). O Gap_{PLI} médio em todas as 13 instâncias foi de apenas $3,51\%$, comprovando que a meta-heurística consegue atingir patamares muito próximos do modelo exato sem o custo computacional proibitivo de formulações inteiras.

Em relação ao tempo de execução entre GRASP e PLI (Figura 1), a maior vantagem a favor do GRASP ocorreu em `p43.atsp`: o Gurobi esgotou o limite de $180,04$ s na árvore de *branch-and-bound*, enquanto o GRASP entregou uma solução a $1,70\%$ do ótimo em apenas $5,34$ s (Com *Union-Find*) e $6,60$ s (Sem *Union-Find*), mais de 33 vezes mais rápido. Em `ft53.atsp` e `br17.atsp`, o GRASP com UF também foi mais rápido que o PLI ($11,46$ s contra $12,37$ s e $0,28$ s contra $0,32$ s). Por outro lado, nas instâncias com mais de 65 vértices (`ft70.atsp`, `ftv64.atsp` e `ftv70.atsp`), o PLI foi mais rápido porque o solver exato podou o espaço eficientemente em poucos nós, enquanto o GRASP executa 1000 iterações de busca local $O(n^3)$.

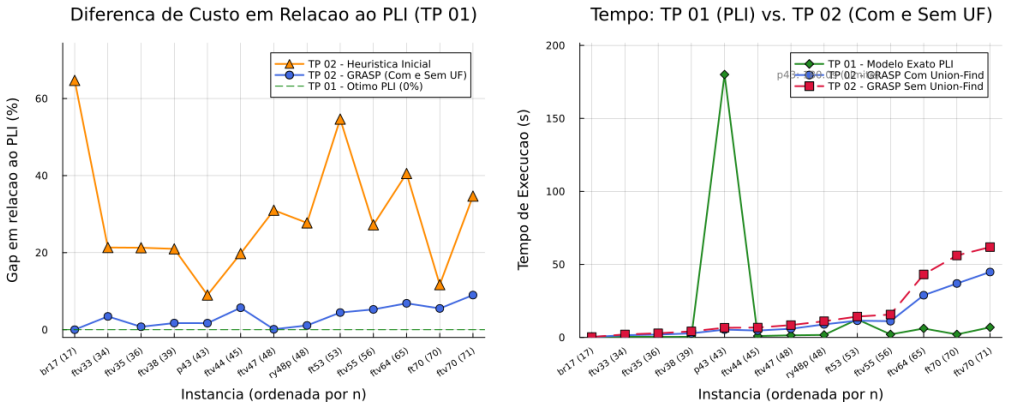


Figura 1: Comparação direta entre o Trabalho 01 (PLI) e o Trabalho 02 (Heurística Inicial e GRASP Com e Sem *Union-Find*) quanto à diferença percentual de custo (esquerda) e ao tempo de execução por instância (direita).

Comparando as duas versões do GRASP (com e sem *Union-Find*, Figura 2), ambas chegam exatamente aos mesmos custos por compartilharem a mesma semente aleatória (`seed = 42`), mas o ganho de tempo proporcionado pelo *Union-Find* é evidente e cresce com o tamanho do grafo. Em instâncias pequenas ($n < 45$), a diferença é modesta, mas para instâncias grandes ($n \geq 53$) a curva sem *Union-Find* descola expressivamente: o *Union-Find* economiza $19,18$ s em `ft70.atsp` ($36,94$ s vs. $56,12$ s, redução de $34,2\%$), $17,06$ s em `ftv70.atsp` ($27,6\%$), $14,23$ s em `ftv64.atsp` ($33,0\%$) e $2,82$ s em `ft53.atsp` ($19,8\%$). No total acumulado das 13 instâncias em 1000

iterações, o tempo total cai de 233,19 s (Sem UF) para 164,63 s (Com UF), uma economia líquida de 68,57 s (29,4%).

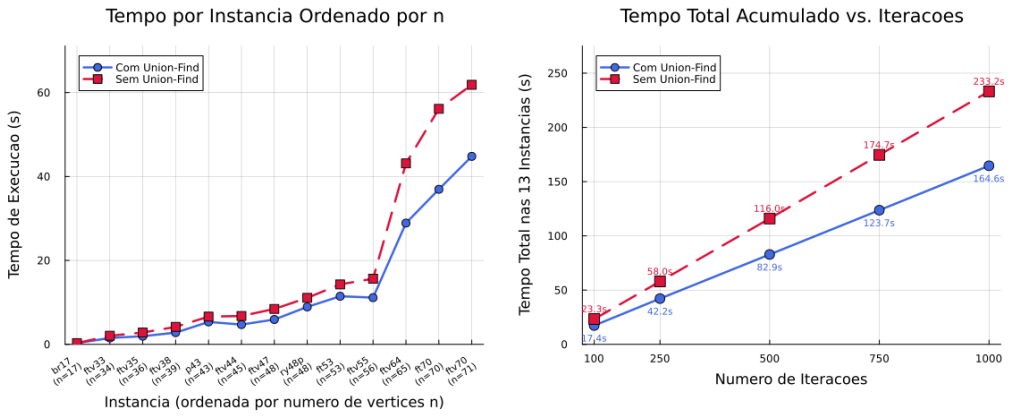


Figura 2: Comparação de tempo de execução Com *Union-Find* vs. Sem *Union-Find* por instância ordenada por n (esquerda) e acumulado por número de iterações (direita).

3.1. Calibração Conjunta dos Parâmetros (Fator Alpha e Iterações)

Para fundamentar cientificamente a escolha dos parâmetros do GRASP perante o professor, desenvolvemos um teste completo de calibração em grade (*grid search*) avaliando 30 combinações: 6 valores de α (0,00; 0,05; 0,15; 0,30; 0,50 e 1,00) cruzados com 5 níveis de iterações (100, 250, 500, 750 e 1000) em todas as 13 instâncias da TSPLIB com semente fixa. Os dados consolidados constam no arquivo `calibracao_resumo.csv` e são ilustrados nas Figuras 3 e 4.

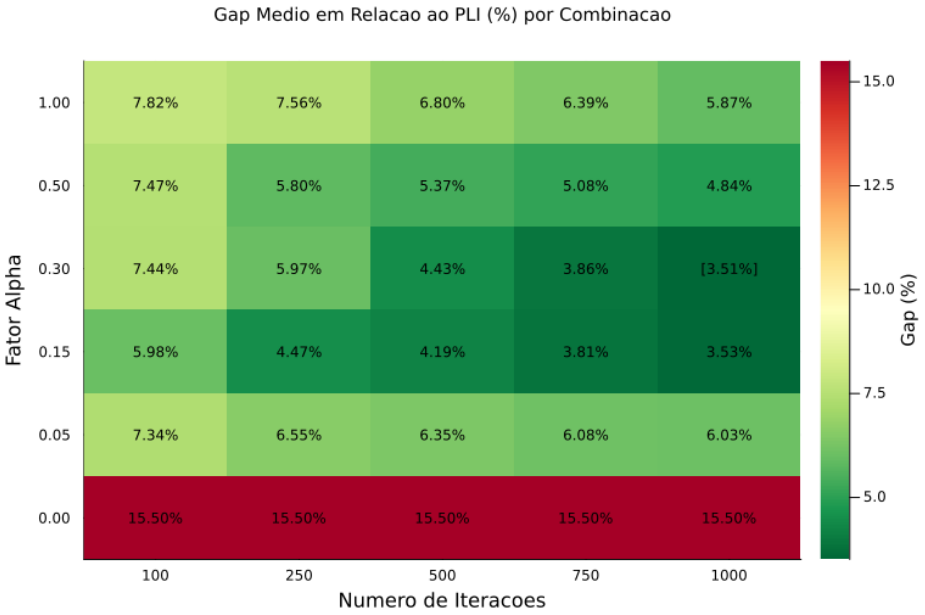


Figura 3: Mapa de calor da calibração conjunta: Gap_{PLI} médio (%) para as 30 combinações de α e número de iterações nas 13 instâncias da TSPLIB. Destaca-se em colchetes a melhor configuração encontrada ($\alpha = 0,30$ com 1000 iterações).

O mapa de calor (Figura 3) e as curvas de convergência (Figura 4) revelam comportamentos fundamentais do algoritmo:

1. **Impacto do $\alpha = 0,00$ (100% Guloso):** A Lista Restrita de Candidatos retém apenas a aresta mais barata, tornando a construção totalmente determinística. Assim, todas as 1000 iterações partem exatamente da mesma solução inicial e convergem imediatamente para o mesmo ótimo local. O Gap_{PLI} estagna permanentemente em 15,50% desde a primeira iteração, comprovando que sem diversificação não há melhora iterativa no GRASP.
2. **Impacto do $\alpha = 1,00$ (100% Aleatório):** A construção torna-se caótica, gerando caminhos iniciais de custo elevadíssimo. Embora a busca local consiga atenuar o erro com muitas iterações (5,87% em 1000 iter.), o tempo computacional explode para 342,67 s (mais que o dobro de $\alpha = 0,15$), pois o operador Or-Opt necessita de dezenas de rodadas corretivas a mais em cada iteração.

3. **Zona Ótima (Sweet Spot):** Os valores intermediários $\alpha \in [0,15;\ 0,30]$ formam o núcleo de melhor desempenho da meta-heurística. Em até 250 iterações, $\alpha = 0,15$ converge mais rapidamente (Gap $= 4,47\%$ vs. $5,97\%$ de $\alpha = 0,30$). Contudo, ao expandir a busca até 1000 iterações, a maior diversificação proporcionada por $\alpha = 0,30$ permite que a busca local escape de bacias de atração restritas em instâncias desafiadoras (como `ft53.atsp`, cujo custo cai de $6950,00$ para $6472,00$, reduzindo o gap de $12,17\%$ para $4,45\%$), atingindo o menor Gap_{PLI} médio global de toda a grade: $3,51\%$ (com melhoria média de $19,08\%$).

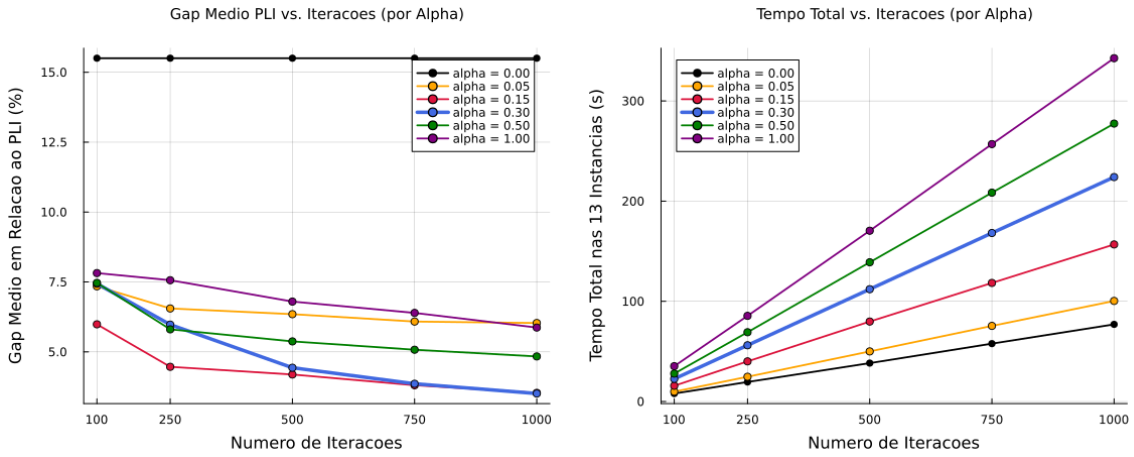


Figura 4: Curvas de calibração do GRASP: evolução do Gap_{PLI} médio por número de iterações para cada α (esquerda) e crescimento do tempo total de execução acumulado nas 13 instâncias (direita).

4. Rotas Encontradas pela Meta-heurística GRASP

br17 (34,00): 1 → 12 → 3 → 14 → 13 → 10 → 11 → 2 → 16 → 15 → 7 → 6 → 4 → 5 → 9 → 8 → 17.

ft53 (6472,00): 1 → 37 → 36 → 41 → 28 → 30 → 29 → 26 → 27 → 42 → 48 → 43 → 47 → 44 → 46 → 45 → 35 → 33 → 34 → 32 → 31 → 4 → 3 → 18 → 17 → 12 → 14 → 11 → 13 → 15 → 21 → 22 → 25 → 24 → 23 → 20 → 19 → 16 → 38 → 40 → 39 → 5 → 2 → 9 → 10 → 8 → 7 → 6 → 52 → 51 → 49 → 50 → 53.

ft70 (39733,00): 1 → 2 → 67 → 66 → 69 → 68 → 64 → 65 → 49 → 44 → 7 → 21 → 18 → 17 → 16 → 24 → 63 → 62 → 60 → 58 → 45 → 28 → 27 → 47 → 46 → 50 → 48 → 56 → 54 → 3 → 5 → 23 → 22 → 25 → 57 → 34 → 32 → 59 → 29 → 55 → 53 → 52 → 51 → 19 → 20 → 26 → 14 → 13 → 11 → 10 → 12 → 9 → 8 → 15 → 6 → 4 → 61 → 35 → 33 → 30 → 31 → 39 → 36 → 43 → 42 → 41 → 37 → 40 → 38 → 70.

ftv33 (1319,00): 1 → 17 → 16 → 15 → 13 → 14 → 10 → 33 → 8 → 9 → 11 → 12 → 32 → 18 → 19 → 20 → 22 → 21 → 23 → 25 → 24 → 27 → 28 → 29 → 30 → 26 → 2 → 4 → 3 → 5 → 7 → 6 → 31 → 34.

ftv35 (1454,00): 1 → 14 → 13 → 6 → 7 → 8 → 35 → 9 → 10 → 12 → 15 → 16 → 17 → 2 → 4 → 3 → 5 → 33 → 31 → 27 → 26 → 25 → 18 → 11 → 34 → 19 → 20 → 21 → 22 → 23 → 24 → 28 → 29 → 30 → 32 → 36.

ftv38 (1537,00): 1 → 2 → 4 → 16 → 7 → 8 → 6 → 9 → 38 → 10 → 11 → 13 → 12 → 17 → 15 → 18 → 19 → 20 → 30 → 29 → 28 → 21 → 14 → 37 → 22 → 23 → 24 → 25 → 26 → 27 → 31 → 32 → 33 → 34 → 35 → 3 → 5 → 36 → 39.

ftv44 (1706,00): 1 → 22 → 20 → 15 → 44 → 12 → 11 → 2 → 3 → 32 → 31 → 30 → 25 → 43 → 24 → 23 → 19 → 17 → 18 → 16 → 13 → 14 → 21 → 8 → 10 → 9 → 41 → 42 → 7 → 40 → 39 → 4 → 5 → 6 → 36 → 33 → 29 → 26 → 27 → 28 → 34 → 35 → 37 → 38 → 45.

ftv47 (1756,00): 1 → 26 → 27 → 2 → 11 → 12 → 9 → 7 → 4 → 25 → 5 → 30 → 31 → 32 → 6 → 10 → 34 → 28 → 29 → 3 → 42 → 44 → 43 → 23 → 41 → 21 → 38 → 39 → 19 → 18 → 13 → 33 → 8 → 24 → 35 → 14 → 47 → 37 → 36 → 15 → 16 → 17 → 46 → 40 → 20 → 45 → 22 → 48.

ftv55 (1698,00): 1 → 34 → 3 → 53 → 26 → 25 → 43 → 22 → 23 → 42 → 51 → 24 → 55 → 28 → 50 → 30 → 27 → 21 → 41 → 20 → 19 → 40 → 39 → 11 → 8 → 33 → 9 → 37 → 10 → 38 → 12 → 52 → 13 → 16 → 17 → 18 → 15 → 14 → 36 → 5 → 7 → 6 → 48 → 32 → 47 → 35 → 2 → 4 → 49 → 45 → 29 → 54 → 44 → 46 → 31 → 56.

ftv64 (1966,00): 1 → 16 → 15 → 48 → 8 → 37 → 9 → 41 → 42 → 10 → 11 → 61 → 47 → 46 → 43 → 44 → 45 → 12 → 13 → 36 → 21 → 22 → 49 → 24 → 50 → 23 → 60 → 26 → 64 → 30 → 33 → 51 → 55 → 29 → 28 → 27 → 20 → 14 → 17 → 18 → 19 → 62 → 2 → 38 → 4 → 39 → 3 → 5 → 40 → 6 → 7 → 58 → 59 → 35 → 54 → 32 → 63 → 53 → 52 → 25 → 31 → 56 → 34 → 57 → 65.

ftv70 (2134,00): 1 → 66 → 2 → 3 → 6 → 32 → 31 → 30 → 25 → 26 → 28 → 7 → 14 → 67 → 15 → 13 → 16 → 17 → 19 → 22 → 23 → 24 → 61 → 34 → 35 → 36 → 70 → 42 → 38 → 64 → 41 → 69 → 47 → 55 → 56 → 57 → 4 → 5 → 9 → 8 → 10 → 12 → 11 → 59 → 62 → 60 → 58 → 53 → 54 → 45 → 46 → 63 → 50 → 49 → 52 → 51 → 68 → 48 → 40 → 39 → 37 → 44 → 43 → 33 → 27 → 20 → 21 → 18 → 65 → 29 → 71.

p43 (599,00): 1 → 5 → 6 → 8 → 7 → 11 → 9 → 10 → 12 → 33 → 32 → 31 → 30 → 18 → 17 → 20 → 19 → 21 → 16 → 13 → 15 → 14 → 34 → 35 → 27 → 25 → 26 → 22 → 23 → 24 → 36 → 37 → 4 → 3 → 29 → 28 → 2 → 38 → 39 → 40 → 41 → 42 → 43.

ry48p (14225,00): 1 → 9 → 40 → 15 → 12 → 11 → 23 → 13 → 25 → 14 → 34 → 41 → 3 → 16 → 22 → 8 → 38 → 31

$\rightarrow 44 \rightarrow 36 \rightarrow 7 \rightarrow 28 \rightarrow 30 \rightarrow 37 \rightarrow 43 \rightarrow 17 \rightarrow 27 \rightarrow 19 \rightarrow 6 \rightarrow 18 \rightarrow 46 \rightarrow 33 \rightarrow 20 \rightarrow 47 \rightarrow 21 \rightarrow 39 \rightarrow 32 \rightarrow 24 \rightarrow 10 \rightarrow 45 \rightarrow 35 \rightarrow 4 \rightarrow 26 \rightarrow 42 \rightarrow 2 \rightarrow 29 \rightarrow 5 \rightarrow 48$.

4.1. Comparação de Caminhos: 3 Exemplos Práticos (Trabalho 01 vs. Trabalho 02)

Para ilustrar como os caminhos encontrados pelo GRASP no Trabalho 02 se comportam frente às rotas exatas do PLI no Trabalho 01, destacamos três casos representativos:

Exemplo 1 – Mesmo Custo Ótimo com Rota Alternativa (`br17.atsp`, $n = 17$, $Z_{\mathbf{PLI}} = 34,00$ e $Z_{\mathbf{GRASP}} = 34,00$):

Trabalho 01 (PLI): $1 \rightarrow 12 \rightarrow \mathbf{14} \rightarrow \mathbf{3} \rightarrow 2 \rightarrow \mathbf{10} \rightarrow \mathbf{11} \rightarrow \mathbf{13} \rightarrow \mathbf{6} \rightarrow \mathbf{16} \rightarrow \mathbf{15} \rightarrow \mathbf{7} \rightarrow \mathbf{5} \rightarrow \mathbf{4} \rightarrow \mathbf{9} \rightarrow \mathbf{8} \rightarrow 17$.

Trabalho 02 (GRASP): $1 \rightarrow 12 \rightarrow \mathbf{3} \rightarrow \mathbf{14} \rightarrow \mathbf{13} \rightarrow \mathbf{10} \rightarrow \mathbf{11} \rightarrow \mathbf{2} \rightarrow \mathbf{16} \rightarrow \mathbf{15} \rightarrow \mathbf{7} \rightarrow \mathbf{6} \rightarrow \mathbf{4} \rightarrow \mathbf{5} \rightarrow \mathbf{9} \rightarrow \mathbf{8} \rightarrow 17$.

Ambos atingiram exatamente o mesmo custo ótimo $(34,00)$, mas explorando rotas distintas no grafo, o que evidencia a existência de múltiplos caminhos ótimos simétricos de custo idêntico.

Exemplo 2 – Reestruturação com Grande Salto de Qualidade (`ft53.atsp`, $n = 53$, $Z_{\mathbf{PLI}} = 6196,00$ vs. $Z_{\mathbf{GRASP}} = 6472,00$):

Heurística Inicial: Custo $9583,00$ (escolhas míopes nas conexões finais).

Trabalho 02 (GRASP $\alpha = 0,30$): Custo $6472,00$ ($\text{Gap}_{\text{PLI}} = 4,45\%$, melhoria de $32,46\%$ sobre a inicial).

Nesta instância, o aumento da diversificação de $\alpha = 0,15$ (que atingia $6950,00$) para $\alpha = 0,30$ permitiu ao GRASP escapar de mínimos locais precoces e encontrar um encadeamento muito mais eficiente dos blocos intermediários $\{21 \ldots 25\}$ e $\{35 \ldots 48\}$.

Exemplo 3 – Convergência Quase Ótima em Instância Difícil (`ftv47.atsp`, $n = 48$, $Z_{\mathbf{PLI}} = 1754,00$ vs. $Z_{\mathbf{GRASP}} = 1756,00$):

Trabalho 01 (PLI): Custo exato $1754,00$ em $1,43$ s.

Trabalho 02 (GRASP): Custo $1756,00$ em $5,92$ s ($\text{Gap}_{\text{PLI}} = 0,11\%$).

O GRASP convergiu para uma rota com diferença de apenas 2 unidades de custo frente ao solver exato, mantendo a mesma macro-sequência de clusters de vértices do modelo ótimo.

5. Representação Gráfica das Rotas (GRASP)

As Figuras 5 a 17 exibem o mapeamento bidimensional das rotas geradas pelo GRASP nas 13 instâncias da TSPLIB, destacando a transição contínua da origem ($s = 1$, verde) ao destino ($t = n$, vermelho).

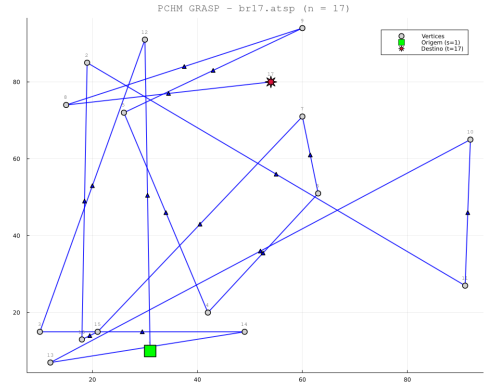


Figura 5: `br17.atsp` ($n = 17$, $Z_{\text{GRASP}} = 34,00$)

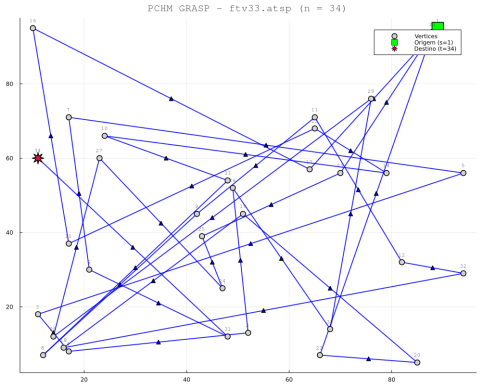


Figura 6: `ftv33.atsp` ($n = 34$, $Z_{\text{GRASP}} = 1319,00$)

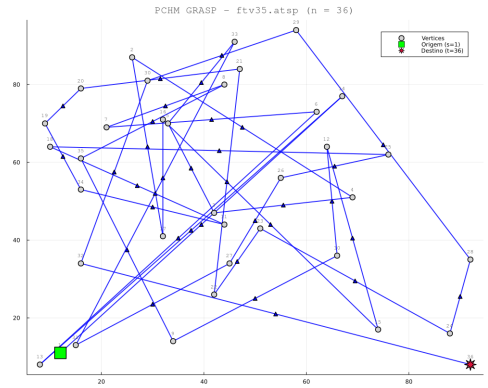


Figura 7: `ftv35.atsp` ($n = 36$, $Z_{\text{GRASP}} = 1454,00$)

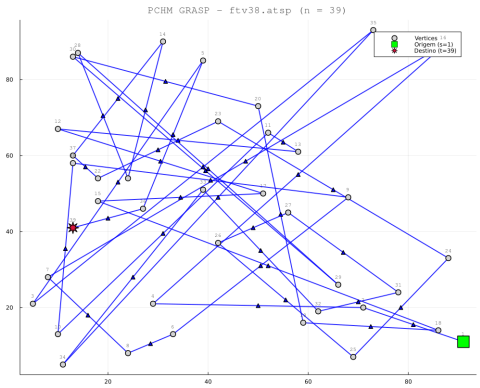


Figura 8: `ftv38.atsp` ($n = 39$, $Z_{\text{GRASP}} = 1537,00$)

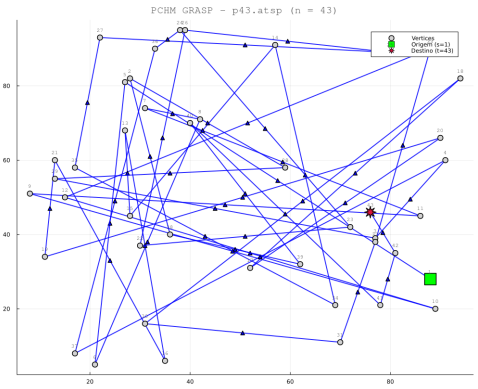


Figura 9: `p43.atsp` ($n = 43$, $Z_{\text{GRASP}} = 599,00$)

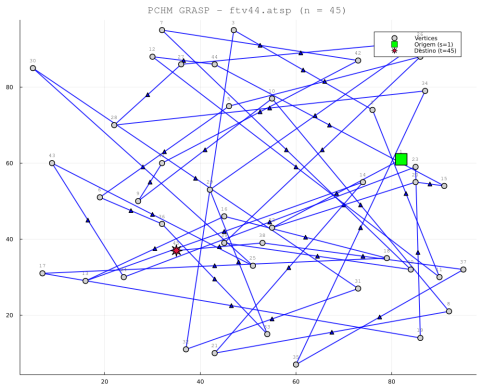


Figura 10: `ftv44.atsp` ($n = 45$, $Z_{\text{GRASP}} = 1706,00$)

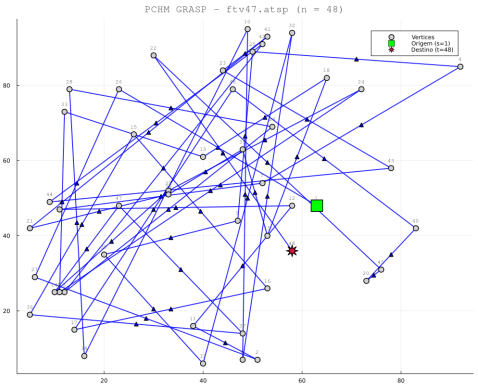


Figura 11: `ftv47.atsp` ($n = 48$, $Z_{\text{GRASP}} = 1756,00$)

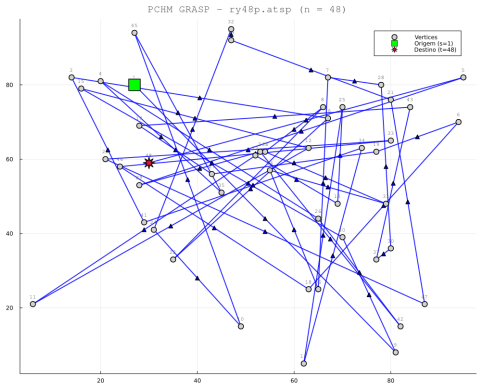


Figura 12: `ry48p.atsp` ($n = 48$, $Z_{\text{GRASP}} = 14225,00$)

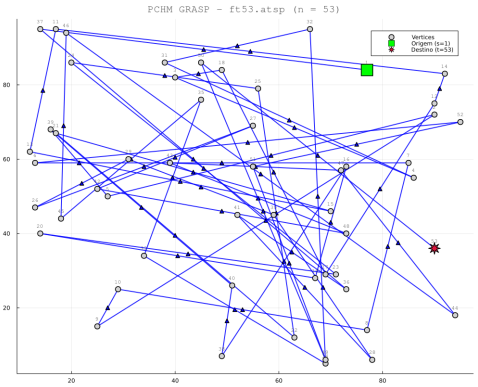


Figura 13: `ft53.atsp` ($n = 53$, $Z_{\text{GRASP}} = 6472,00$)

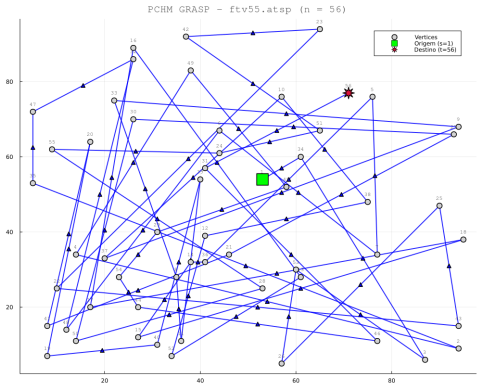


Figura 14: `ftv55.atsp` ($n = 56$, $Z_{\text{GRASP}} = 1698,00$)

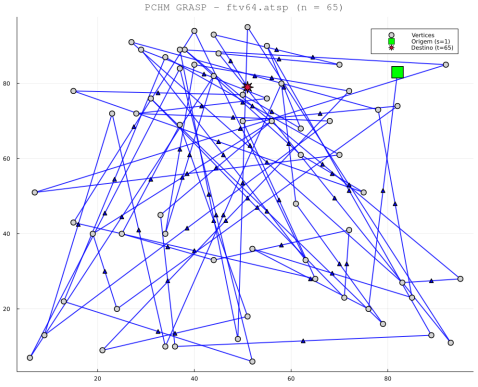


Figura 15: `ftv64.atsp` ($n = 65$, $Z_{\text{GRASP}} = 1966,00$)

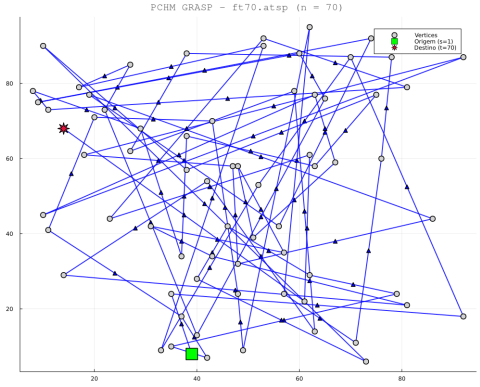


Figura 16: `ft70.atsp` ($n = 70$, $Z_{\text{GRASP}} = 39733,00$)

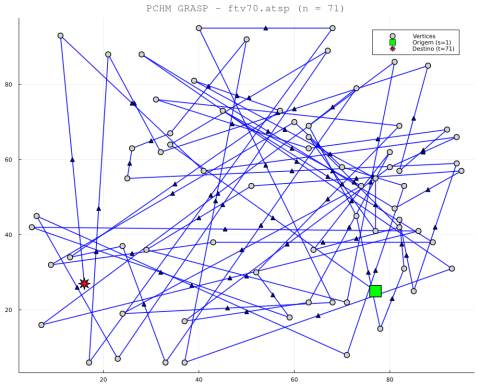


Figura 17: `ftv70.atsp` ($n = 71$, $Z_{\text{GRASP}} = 2134,00$)

6. Declaração de Uso de Inteligência Artificial (Atividade 3.1)

Foi utilizada para revisar a redação do relátorio, pesquisas de artigo e dúvidas para debugar o código da metaheuricas. A IA utlizada foi o Gemini.